EXPERIMENT-13: React weather chart service

Create a service in React that fetches weather information from openweathermap.org and displays the current and historical weather information using graphical representation with chart.js.

Objective : Fetch weather data, transform it for chart display, and render history in a chart component.

Tools : React, OpenWeatherMap API, Chart.js, Fetch API

Outcome : A React dashboard showing current weather and historical trends in a chart.

Answer:

Fetch weather data in a React useEffect hook, transform it into chart arrays, and render a line chart component.

File Name : weather_app.html

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Weather Dashboard</title>

<script crossorigin src="https://unpkg.com/react@18.2.0/umd/react.production.min.js"></script>
<script crossorigin src="https://unpkg.com/react-dom@18.2.0/umd/react-dom.production.min.js"></script>
<script crossorigin src="https://unpkg.com/@babel/standalone@7.23.5/babel.min.js"></script>
<script src="https://cdn.jsdelivr.net/npm/chart.js@4.4.0/dist/chart.umd.min.js"></script>

<style>
 *{box-sizing:border-box}
 body{margin:0;font-family:Arial,sans-serif;background:#eaf2f8;color:#222}
 header{background:#2c3e50;color:#fff;padding:14px 24px;font-size:20px;font-weight:bold}
 .container{max-width:900px;margin:25px auto;padding:0 15px}
 .card{background:#fff;padding:20px;border-radius:8px;box-shadow:0 2px 6px rgba(0,0,0,.1);margin-bottom:20px}
 .row{display:flex;gap:10px;flex-wrap:wrap;align-items:center}
 input[type=text],input[type=password]{padding:9px;border:1px solid #ccc;border-radius:4px;font-size:14px;flex:1;min-width:150px}
```

```
  button{background:#1abc9c;color:#fff;border:0;padding:10px 18px;border-
      radius:4px;cursor:pointer;font-size:15px}
  button:hover{background:#16a085}
  .error{background:#fdecea;color:#c0392b;padding:10px;border-radius:4px;margin-bottom:15px}
  .note{background:#fff8e1;color:#8a6d3b;padding:10px;border-radius:4px;margin-
      bottom:15px;font-size:14px}
  .current{display:flex;align-items:center;gap:20px;flex-wrap:wrap}
  .temp{font-size:48px;font-weight:bold}
  .grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(130px,1fr));gap:10px;margin-
      top:15px}
  .stat{background:#f4f6f8;padding:10px;border-radius:6px;text-align:center}
  .stat b{display:block;font-size:18px;margin-top:4px}
  small{color:#666}
</style>
</head>
<body>
<header>🌦️ Weather Dashboard</header>
<div class="container"><div id="root"></div></div>

<script type="text/babel">
const { useState, useEffect, useRef } = React;

const BASE = 'https://api.openweathermap.org';

// ---------- Fetch helper ----------
async function getJSON(url) {
  const res = await fetch(url);
  const data = await res.json();
  if (!res.ok) throw new Error(data.message || 'Request failed (' + res.status + ')');
  return data;
}

// ---------- Demo data (no API key needed) ----------
function demoData(city) {
  const base = 22 + Math.round(Math.random() * 8);
  const current = {
```

```
    name: city, sys: { country: 'DEMO' },
    main: { temp: base, feels_like: base + 1, humidity: 60, pressure: 1012 },
    wind: { speed: 3.4 }, weather: [{ description: 'sample data', icon: '02d' }],
  };
  const labels = [], temps = [], humidity = [];
  for (let d = 5; d >= 1; d--) {
    labels.push(new Date(Date.now() - d * 864e5).toLocaleDateString());
    temps.push(base + Math.round(Math.random() * 6 - 3));
    humidity.push(50 + Math.round(Math.random() * 30));
  }
  return { current, history: { labels, temps, humidity, mode: 'Demo history (sample data)' } };
}

// ---------- Transform API data -> chart arrays ----------
async function fetchWeather(city, key) {
  // 1) Current weather (free plan)
  const current = await getJSON(
    BASE + '/data/2.5/weather?q=' + encodeURIComponent(city) + '&units=metric&appid=' + key);

  // 2) Historical weather: One Call 3.0 "timemachine" (one call per past day)
  const { lat, lon } = current.coord;
  const now = Math.floor(Date.now() / 1000);
  try {
    const days = [5, 4, 3, 2, 1];
    const results = await Promise.all(days.map(d =>
      getJSON(BASE + '/data/3.0/onecall/timemachine?lat=' + lat + '&lon=' + lon +
            '&dt=' + (now - d * 86400) + '&units=metric&appid=' + key)));
    return {
      current,
      history: {
        labels: results.map(r => new Date(r.data[0].dt * 1000).toLocaleDateString()),
        temps: results.map(r => r.data[0].temp),
        humidity: results.map(r => r.data[0].humidity),
        mode: 'Historical data - last 5 days',
      },
    };
```

```
  } catch (e) {
    // Fallback: 5-day forecast (free plan) when history is not enabled for the key
    const f = await getJSON(
      BASE + '/data/2.5/forecast?q=' + encodeURIComponent(city) + '&units=metric&appid=' + key);
    return {
      current,
      history: {
        labels: f.list.map(i => i.dt_txt.slice(5, 16)),
        temps: f.list.map(i => i.main.temp),
        humidity: f.list.map(i => i.main.humidity),
        mode: 'Forecast trend (history needs One Call 3.0 subscription)',
      },
      warning: 'Historical API not available for this key (' + e.message +
            '). Showing the free 5-day forecast trend instead.',
    };
  }
}

// ---------- Chart.js line chart component ----------
function WeatherChart({ labels, temps, humidity, title }) {
  const canvasRef = useRef(null);
  useEffect(() => {
    const chart = new Chart(canvasRef.current, {
      type: 'line',
      data: {
        labels,
        datasets: [
          { label: 'Temperature (°C)', data: temps, borderColor: '#e74c3c',
            backgroundColor: 'rgba(231,76,60,.15)', fill: true, tension: 0.3, yAxisID: 'y' },
          { label: 'Humidity (%)', data: humidity, borderColor: '#3498db',
            tension: 0.3, yAxisID: 'y1' },
        ],
      },
      options: {
        responsive: true,
        plugins: { title: { display: true, text: title } },
```

```
        scales: {
          y: { position: 'left', title: { display: true, text: 'Temperature (°C)' } },
          y1: { position: 'right', grid: { drawOnChartArea: false },
                title: { display: true, text: 'Humidity (%)' } },
        },
      },
    });
    return () => chart.destroy();   // cleanup before re-draw
  }, [labels, temps, humidity, title]);
  return <canvas ref={canvasRef}></canvas>;
}

// ---------- Current weather card ----------
function CurrentWeather({ w }) {
  return (
    <div className="card">
      <h2 style={{ marginTop: 0 }}>Current weather in {w.name}, {w.sys.country}</h2>
      <div className="current">
        <img alt="icon" width="80" height="80"
             src={'https://openweathermap.org/img/wn/' + w.weather[0].icon + '@2x.png'} />
        <div className="temp">{Math.round(w.main.temp)}°C</div>
        <div style={{ textTransform: 'capitalize' }}>{w.weather[0].description}</div>
      </div>
      <div className="grid">
        <div className="stat">Feels like<b>{Math.round(w.main.feels_like)}°C</b></div>
        <div className="stat">Humidity<b>{w.main.humidity}%</b></div>
        <div className="stat">Pressure<b>{w.main.pressure} hPa</b></div>
        <div className="stat">Wind<b>{w.wind.speed} m/s</b></div>
      </div>
    </div>
  );
}

// ---------- App ----------
function App() {
  const [apiKey, setApiKey] = useState(localStorage.getItem('owmKey') || '');
```

```
const [cityInput, setCityInput] = useState('Hyderabad');
const [demo, setDemo] = useState(false);
const [query, setQuery] = useState({ city: 'Hyderabad', n: 0 }); // changes on Search click
const [data, setData] = useState(null);
const [error, setError] = useState('');
const [loading, setLoading] = useState(false);

// Fetch data whenever the search query changes (useEffect)
useEffect(() => {
  if (query.n === 0) return;          // do nothing on first render
  let cancelled = false;
  async function load() {
    setLoading(true); setError('');
    try {
      const result = demo ? demoData(query.city) : await fetchWeather(query.city, apiKey);
      if (!cancelled) setData(result);
    } catch (e) {
      if (!cancelled) { setData(null); setError(e.message); }
    }
    if (!cancelled) setLoading(false);
  }
  load();
  return () => { cancelled = true; };
}, [query]);

const search = () => {
  if (!demo && !apiKey.trim()) return setError('Enter your OpenWeatherMap API key or tick Demo
     mode');
  if (!cityInput.trim()) return setError('Enter a city name');
  localStorage.setItem('owmKey', apiKey.trim());
  setQuery({ city: cityInput.trim(), n: query.n + 1 });
};

return (
  <div>
    <div className="card">
```

```
      <div className="row">
        <input type="password" placeholder="OpenWeatherMap API key"
               value={apiKey} onChange={e => setApiKey(e.target.value)} />
        <input type="text" placeholder="City"
               value={cityInput} onChange={e => setCityInput(e.target.value)}
               onKeyDown={e => e.key === 'Enter' && search()} />
        <button onClick={search}>Get Weather</button>
      </div>
      <div style={{ marginTop: 10 }}>
        <label><input type="checkbox" checked={demo} onChange={e => setDemo(e.target.checked)}
      />
          {' '}Demo mode (sample data, no API key)</label>
      </div>
      <small>Get a free key at openweathermap.org/api (new keys can take up to 2 hours to
      activate).</small>
    </div>

    {error && <div className="error">⚠ {error}</div>}
    {loading && <div className="card">Loading...</div>}
    {data && data.warning && <div className="note">{data.warning}</div>}
    {data && <CurrentWeather w={data.current} />}
    {data && (
      <div className="card">
        <WeatherChart
          labels={data.history.labels}
          temps={data.history.temps}
          humidity={data.history.humidity}
          title={data.history.mode + ' - ' + data.current.name} />
      </div>
    )}
  </div>
 );
}

ReactDOM.createRoot(document.getElementById('root')).render(<App />);
</script>
```

```
</body>
</html>
```